

Introduction to Antivirus

Aim:

To understand antivirus software, its features, and different malware detection techniques such as static, dynamic, and heuristic analysis.

Welcome to Intro to AV

Antivirus () software is one of the essential host-based security solutions available to detect and prevent malware attacks within the end-user's machine. software consists of different modules, features, and detection techniques, which are discussed in this room.

As a red teamer or pentester, it is essential to be familiar with and understand how software and its detection techniques work. Once this knowledge is acquired, it will be easier to work toward evasion techniques.



Learning Objectives

- What is Antivirus software?
- Antivirus detection approaches
- Enumerate installed software in the target machine
- Test in a simulated environment

Room prerequisites

- General knowledge of host-based detection solutions; check [The Lay of the Land](#) room for more information.
- General experience with Hashing crypto; check the [Hashing - Crypto 101](#) room for more information.
- Basic knowledge of Yara Rules; check the [Yara](#) room for more information.

Answer the questions below
Let's get started!

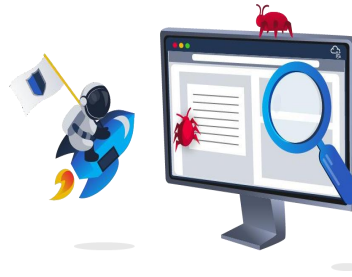
No Answer Needed

Task 2: Antivirus Software

What is AV software?

Antivirus () software is an extra layer of security that aims to detect and prevent the execution and spread of malicious files in a target operating system.

It is a host-based application that runs in real-time (in the background) to monitor and check the current and newly downloaded files. The software inspects and decides whether files are malicious using different techniques, which will be covered later in this room.



Interestingly, the first antivirus software was designed solely to detect and remove [computer viruses \(opens in new tab\)](#). Nowadays, that has changed; modern antivirus applications can detect and remove computer viruses as well other harmful files and threats.

What does software look for?

Traditional software looks for **malware** with predefined malicious patterns or signatures. Malware is harmful software whose primary goal is to cause damage to a target machine, including but not limited to:

- Gain full access to a target machine.
- Steal sensitive information such as passwords.
- Encrypt files and cause damage to files.
- Inject other malicious software or unwanted advertisements.
- Used the compromised machine to perform further attacks such as botnet attacks.

vs other security products



In addition to software, other host-based security solutions provide real-time protection to endpoint devices. Endpoint Detection and Response () is a security solution that provides real-time protection based on behavioral analytics. An antivirus application performs scanning, detecting, and removing malicious files. On the other hand, monitors various security checks in the target machine, including file activities, memory, network connections, Windows registry, processes, etc.

Modern Antivirus products are implemented to integrate the traditional Antivirus features and other advanced functionalities (similar to functionalities) into one product to provide comprehensive protection against digital threats. For more information about Host-based security solutions, we suggest visiting the room: [The Lay of the Land](#).

software in the past and present

McAfee Associates, Inc. started the first software implementation in 1987. It was called "VirusScan," and its main goal at that time was to remove a virus named "Brain" that infected John McAfee's computer. Later, other companies joined in the battle against viruses. software was called scanners, and they were command-line software that searched for malicious patterns in files.

Since then, things have changed. software nowadays uses a Graphical User Interface () to perform scans for malicious files and other tasks. Malware programs have also expanded in scope and now target victims on Windows and other operating systems. Modern software supports most devices and platforms, including Windows, , macOS, Android, and iOS. Modern software has improved and become more intelligent and sophisticated, as they pack a bundle of versatile features, including Antivirus, Anti-Exploit, , Encryption tool, etc.

We will be discussing some features in the next task.

Answer the questions below

What does AV mean?

Answer : Antivirus

Which PC Antivirus vendor implemented the first AV software on the market?

Answer: McAfee

Antivirus software is a _____-based security solution.

Answer: Host

Task 3: Antivirus Features

Antivirus Engines

An engine is responsible for finding and removing malicious code and files. Good software implements an effective and solid core that accurately and quickly analyzes malicious files. Also, It should handle and support various file types, including archive files, where it can self-extract and inspect all compressed files.

Most products share the same common features but are implemented differently, including but not limited to:

- Scanner
- Detection techniques
- Compressors and Archives
- Unpackers
- Emulators

Scanner

The scanner feature is included in most products: software runs and scans in real-time or on-demand. This feature is available in the or through the command prompt. The user can use it whenever required to check files or directories. The scanning feature must support the most known malicious file types to detect and remove the threat. In addition, it also may support

other types of scanning depending on the software, including vulnerabilities, emails, Windows memory, and Windows Registry.

Detection techniques

An detection technique searches for and detects malicious files; different detection techniques can be used within the engine, including:

- Signature-based detection is the traditional technique that looks for predefined malicious patterns and signatures within files.
- Heuristic detection is a more advanced technique that includes various behavioral methods to analyze suspicious files.
- Dynamic detection is a technique that includes monitoring the system calls and APIs and testing and analyzing in an isolated environment.

We will cover these techniques in the next task. A good engine is accurate and quickly detects malicious files with fewer false-positive results. We will showcase several products that provide inaccurate results and misclassify a file.

Compressors and Archives

The "Compressors and Archives" feature should be included in any software. It must support and be able to deal with various system file types, including compressed or archived files: ZIP, TGZ, 7z, XAR, RAR, etc. Malicious code often tries to evade host-based security solutions by hiding in compressed files. For this reason, software must decompress and scan through all files before a user opens a file within the archive.

(Portable Executable) Parsing and Unpackers

Malware hides and packs its malicious code by compressing and encrypting it within a payload. It decompresses and decrypts itself during runtime to make it harder to perform . Thus, software must be able to detect and unpack most of the known (UPX, Armadillo, ASPack, etc.) before the runtime for .

Malware developers use various techniques, such as Packing, to shrink the size and change the malicious file's structure. Packing compresses the original executable file to make it harder to analyze. Therefore, software must have an unpacker feature to unpack protected or compressed executable files into the original code.

Another feature that software must have is Windows Portable Executable () header parser. Parsing of executable files helps distinguish malicious and legitimate software (.exe files). The file format in Windows (32 and 64 bits) contains various information and resources, such as object code, DLLs, icon files, font files, and core dumps.

Emulators

An emulator is an Antivirus feature that does further analysis on suspicious files. Once an emulator receives a request, the emulator runs the suspect (exe, , PDF, etc.) files in a virtualized and controlled environment. It monitors the executable files' behavior during the execution, including the Windows APIs calls, Registry, and other Windows files. The following are examples of the artifacts that the emulator may collect:

- calls

- Memory dumps
- Filesystem modifications
- Log events
- Running processes
- Web requests

An emulator stops the execution of a file when enough artifacts are collected to detect malware.

Other common features

The following are some common features found in products:

- A self-protection driver to guard against malware attacking the actual .
- and network inspection functionality.
- Command-line and graphical interface tools.
- A daemon or service.
- A management console.

Answer the questions below

Which AV feature analyzes malware in a safe and isolated environment?

Answer: Emulator

An _____ feature is a process of restoring or decrypting the compressed executable files to the original.

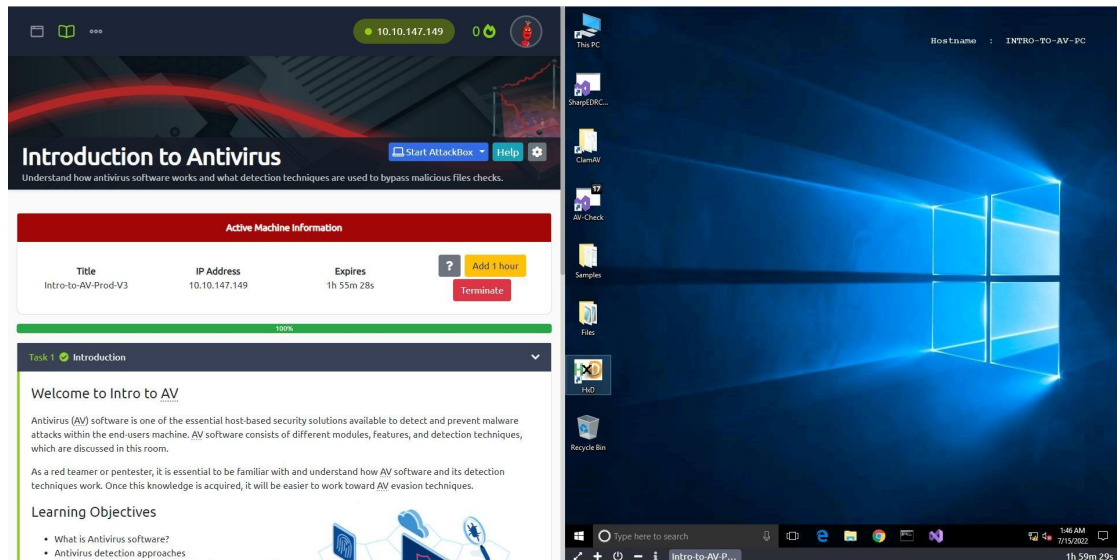
Answer: unpacker

Read the above to proceed to the next task, where we discuss the AV detection techniques.

No Answer Needed

Task 4 Deploy the VM

We have provided a Windows machine 10 Pro to complete this room. The can be accessed through the in-browser feature. Once the is deployed, your browser should split into two windows, as the following:



If the browser doesn't split, you can select the "Show Split View" button located at the top of your browser.



You can also connect via , make sure you deploy the AttackBox or connect to the .

Use the following credentials below.

Machine IP: MACHINE_IP Username: Password: TryHackM3

Connect to the via the client

user@machine\$ xfreerdp /v:MACHINE_IP /u:thm
/p:TryHackM3

Answer the questions below

Once you've deployed the VM, it will take a few minutes to boot up. Then, progress to the next task!

No Answer Needed

Task 5: AV Static Detection

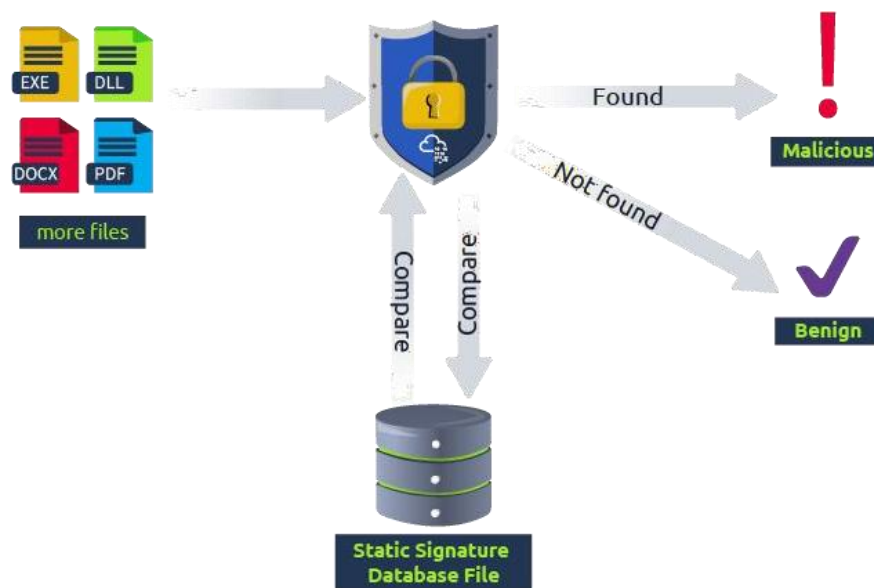
Generally speaking, detection can be classified into three main approaches:

1. Static Detection
2. Dynamic Detection
3. Heuristic and Behavioral Detection

Static Detection

A static detection technique is the simplest type of Antivirus detection, which is based on predefined signatures of malicious files. Simply, it uses pattern-matching techniques in the detection, such as finding a unique string, CRC (Checksums), sequence of bytecode/Hex values, and Cryptographic hashes (, SHA1, etc.).

It then performs a set of comparisons between existing files within the operating system and a database of signatures. If the signature exists in the database, then it is considered malicious. This method is effective against static malware.



In this task, we will be using a signature-based detection method to see how antivirus products detect malicious files. It is important to note that this technique works against known malicious files only with pre-generated signatures in a database. Thus, the database needs to be updated from time to time.

We will use the **ClamAV** antivirus software to demonstrate how signature-based detection identifies malicious files. The ClamAV software is pre-installed in the provided , and we can access it in the following path: **c:\Program Files\ClamAV\clamsan.exe**. We will also scan a couple of malware samples, which can be found on the desktop. The Malware samples folder contains the following files:

1. EICAR is a test file containing ASCII strings used to test software's effectiveness instead of real malware that could damage your machine. For more information, you may visit the official EICAR website [here \(opens in new tab\)](#).
2. **Backdoor 1** is a C# program that uses a well-known technique to establish a reverse connection, including creating a process and executing a Framework shellcode.h
3. **Backdoor 2** is a C# program that uses process injection and encryption to establish a reverse connection, including injecting a shellcode into an existing and running process.
4. **-Check** is a C# program that enumerates software in a target machine. Note that this file is not malicious. We will discuss this tool in more detail in task 6.
5. **notes.txt** is a text file that contains a command line. Note that this file is not malicious.

ClamAV comes with its database, and during the installation, we need to download the recently updated version. Let's try to scan the Malware sample folder using the **clamscan.exe** binary and check how ClamAV performs against these samples.

Command Prompt

```
c:\>"c:\Program Files\ClamAV\clamscan.exe"
c:\Users\thm\Desktop\Samples
Loading:  22s, ETA:  0s [=====>]  8.61M/8.61M sigs
Compiling: 4s, ETA:  0s [=====>]  41/41 tasks

C:\Users\thm\Desktop\Samples\AV-Check.exe: OK
C:\Users\thm\Desktop\Samples\backdoor1.exe: Win.Malware.Swrort-9872015-0 FOUND
C:\Users\thm\Desktop\Samples\backdoor2.exe: OK
C:\Users\thm\Desktop\Samples\eicar.com: Win.Test.EICAR_HDB-1 FOUND
C:\Users\thm\Desktop\Samples\notes.txt: OK
```

The above output shows that ClamAV software correctly analyzed and flagged two of our tested files (EICAR, backdoor1, -Check, and notes.txt) as malicious. However, it incorrectly identified the backdoor2 as non-malicious while it does.

You can run **clamscan.exe --debug <file_to_scan>**, and you will see all modules loaded and used during the scanning. For example, it uses the unpacking method to split the files and look for a predefined malicious sequence of bytecode values, and that is how it was able to detect the C# backdoor 1. The bytecode value of the Metasploit shellcode used in backdoor 1 was previously identified and added to ClamAV's database.

However, backdoor 2 uses an encryption technique (XOR) for the Metasploit shellcode, resulting in different sequences of bytecode values that it doesn't find in the ClamAV database.

While the ClamAV was able to detect the EICAR.COM test file as malicious using the md5 signature-based technique. To confirm this, we can re-scan the EICAR.COM test file again in debug mode (--debug). At some point in the output, you will see the following message:

LibClamAV debug: FP SIGNATURE:

44d88612fea8a8f36de82e1278abb02f:68:Win.Test.EICAR_HDB-1 # Name: eicar.com, Type:
CL_TYPE_TEXT_ASCII

Now let's generate the value of the EICAR.COM if it matches what we see in the previous message from the output. We will be using the sigtool for that:

Command Prompt

```
c:\>"c:\Program Files\ClamAV\sigtool.exe" --md5  
c:\Users\thm\Desktop\Samples\eicar.com  
44d88612fea8a8f36de82e1278abb02f:68:eicar.com
```

If you closely check the generated value, **44d88612fea8a8f36de82e1278abb02f**, it matches.

Create Your Own Signature Database

One of ClamAV's features is creating your own database, allowing you to include items not found in the official ClamAV database. Let's try to create a signature for Backdoor 2, which ClamAV already missed, and add it to a database. The following are the required steps:

1. Generate an signature for the file.
2. Add the generated signature into a database with the extension ".hdb".
3. Re-scan the ClamAV against the file using our new database.

First, we will be using the **sigtool** tool, which is included in the ClamAV suite, to generate an hash of **backdoor2.exe** using the **--** argument.

Generate an hash

```
C:\Users\thm\Desktop\Samples>"c:\Program  
Files\ClamAV\sigtool.exe" --md5 backdoor2.exe  
75047189991b1d119fdb477fef333ceb:6144:backdoor2.exe
```

As shown in the output, the generated hash string contains the following structure: **Hash:Size-in-byte:FileName**. Note that ClamAV uses the generated value in the comparison during the scan.

Now that we have the hash, now let's create our own database. We will use the **sigtool** tool and save the output into a file using the **> .hdb** as follows,

Generate our new database

```
C:\Users\thm\Desktop\Samples>"c:\Program  
Files\ClamAV\sigtool.exe" --md5 backdoor2.exe > thm.hdb
```

As a result, a **.hdb** file will be created in the current directory that executes the command.

We already know that ClamAV did not detect the backdoor2.exe using the official database! Now, let's re-scan it using the database we created, **.hdb**, and see the result!

Re-scanning backdoor2.exe using the new database!

```
C:\Users\thm\Desktop\Samples>"c:\Program  
Files\ClamAV\clamscan.exe" -d thm.hdb backdoor2.exe  
Loading: 0s, ETA: 0s [=====>] 1/1 sigs  
Compiling: 0s, ETA: 0s [=====>] 10/10 tasks
```

```
C:\Users\thm\Desktop\Samples\backdoor2.exe: backdoor2.exe.UNOFFICIAL FOUND
```

As we expected, the **ClamAV** tool flagged the **backdoor2.exe** binary as malicious based on the database we provided. As a practice, add the AV-Check.exe's MD5 signature into the same database we already created, then check whether ClamAV can flag AV-Check.exe as malicious.

Yara Rules for Static Detection

One of the tools that help in static detection is [Yara \(opens in new tab\)](#). Yara is a tool that allows malware engineers to classify and detect malware. Yara uses rule-based detection, so in order to detect new malware, we need to create a new rule. ClamAV can also deal with Yara rules to detect malicious files. The rule will be the same as in our database in the previous section.

To create a rule, we need to examine and analyze the malware; based on the findings, we write a rule. Let's take AV-Check.exe as an example and write a rule for it.

First, let's analyze the file and list all human-readable strings in the binary using the strings tool. As a result, we will see all functions, variables, and nonsense strings. But, if you look closely, we can use some of the unique strings in our rules to detect this file in the future. The

AV-Check uses a program database (.pdb), which contains a type and symbolic debugging information of the program during the compiling.

Command Prompt

```
C:\Users\thm\Desktop\Samples>strings AV-Check.exe |  
findstr pdb  
C:\Users\thm\source\repos\AV-Check\AV-Check\obj\Debug\AV-Check.pdb
```

We will use the path in the previous command's output as our unique string example in the Yara rule that we will create. The signature could be something else in the real world, such as Registry keys, commands, etc. If you are not familiar with Yara, then we suggest checking the [Yara room](#). The following is Yara's rule that we will use in our detection:

```
rule thm_demo_rule {  
    meta:  
        author = "THM: Intro-to-AV-Room"  
        description = "Look at how the Yara rule works with ClamAV"  
    strings:  
        $a = "C:\\Users\\thm\\source\\repos\\AV-Check\\AV-  
Check\\obj\\Debug\\AV-Check.pdb"  
    condition:  
        $a  
}
```

Let's explain this Yara's rule a bit more.

- The rule starts with **rule thm_demo_rule**, which is the name of our rule. ClamAV uses this name if a rule matches.
- The metadata section, which is general information, contains the author and description, which the user can fill.
- The strings section contains the strings or bytecode that we are looking for. We are using the C# program's database path in this case. Notice that we add an extra `\` in that path to escape the special character, so it does not break the rule.
- In the condition section, we specify if the defined string is found in the string section, then flag the file.

Note that Yara rules must store in a **.yara** extension file for ClamAV to deal with it. Let's re-scan the **c:\Users\Desktop\Samples** folder again using the Yara rule we created. You can find a copy of the Yara rule on the desktop at **c:\Users\Desktop\Files\-demo-1.yara**.

Scanning using the Yara rule

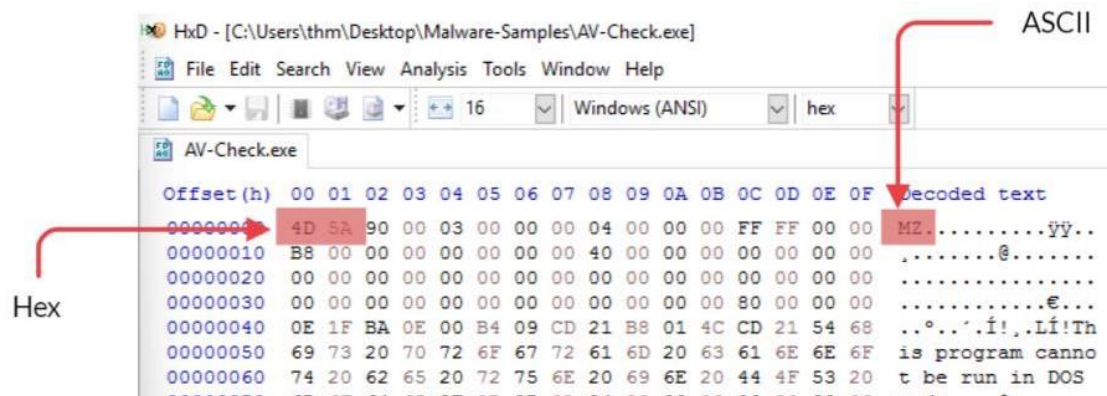
```
C:\Users\thm>"c:\Program Files\ClamAV\clamscan.exe" -d
Desktop\Files\thm-demo-1.yara Desktop\Samples
Loading: 0s, ETA: 0s [=====>] 1/1 sigs
Compiling: 0s, ETA: 0s [=====>] 40/40 tasks

C:\Users\thm\Desktop\Samples\AV-Check.exe: YARA.thm_demo_rule.UNOFFICIAL
FOUND
C:\Users\thm\Desktop\Samples\backdoor1.exe: OK
C:\Users\thm\Desktop\Samples\backdoor2.exe: OK
C:\Users\thm\Desktop\Samples\eicar.com: OK
C:\Users\thm\Desktop\Samples\notes.txt: YARA.thm_demo_rule.UNOFFICIAL FOUND
```

As a result, ClamAV can detect the **-Check.exe** binary as malicious based on the Yara rule we provide. However, ClamAV gave a false-positive result where it flagged the **notes.txt** file as malicious. If we open the **notes.txt** file, we can see that the text contains the same path we specified in the rule.

Let's improve our Yara rule to reduce the false-positive result. We will be specifying the file type in our rule. Often, the types of a file can be identified using magic numbers, which are the first two bytes of the binary. For example, [executable files \(opens in new tab\)](#) (.exe) always start with the ASCII "MZ" value or "4D 5A" in hex.

To confirm this, let's use the [HxD \(opens in new tab\)](#) application, which is a freeware Hex Editor, to examine the -Check.exe binary and see the first two bytes. Note that the HxD is already available in the provided VM.



Knowing this will help improve the detection, let's include this in our Yara rule to flag only the .exe files that contain our signature string as malicious. The following is the improved Yara rule:

```
rule thm_demo_rule {
  meta:
```

```
        author = "THM: Intro-to-AV-Room"
        description = "Look at how the Yara rule works with ClamAV"
    strings:
        $a = "C:\\Users\\thm\\source\\repos\\AV-Check\\AV-
Check\\obj\\Debug\\AV-Check.pdb"
        $b = "MZ"
    condition:
        $b at 0 and $a
}
```

In the new Yara rule, we defined a unique string (\$b) equal to the MZ as an identifier for the .exe file type. We also updated the condition section, which now includes the following conditions:

1. If the string "MZ" is found at the 0 location, the file's beginning.
2. If the unique string (the path) occurs within the binary.
3. In the condition section, we used the **AND** operator for both definitions in 1 and 2 are found, then we have a match.

You can find the updated rule in **Desktop\\Files\\-demo-2.yara**. Now that we have our updated Yara rule, now let's try it again.

Scanning using the Yara rule

```
C:\Users\thm>"c:\Program Files\ClamAV\clamscan.exe" -d
Desktop\Files\thm-demo-2.yara Desktop\Samples
Loading: 0s, ETA: 0s [=====>] 1/1 sigs
Compiling: 0s, ETA: 0s [=====>] 40/40 tasks

C:\Users\thm\Desktop\Samples\AV-Check.exe: YARA.thm_demo_rule.UNOFFICIAL
FOUND
C:\Users\thm\Desktop\Samples\backdoor1.exe: OK
C:\Users\thm\Desktop\Samples\backdoor2.exe: OK
C:\Users\thm\Desktop\Samples\eicar.com: OK
C:\Users\thm\Desktop\Samples\notes.txt: OK
```

The output shows we improved our Yara rule to reduce the false-positive results. That was a simple example of how software works. Thus, software vendors work hard to fight against malware and improve their products and database to enhance the performance and accuracy of results.

The drawback of the signature-based detection is that files will have a different hash value if the binary is modified. Therefore, it is easy for someone to bypass signature-based detection

techniques if they know what software looks for and how to analyze binaries, as shown in later rooms.

Answer the questions below

What is the **sigtool** tool output to generate an MD5 of the **AV-Check.exe** binary?

Answer: f4a974b0cf25dca7fbce8701b7ab3a88:6144:AV-Check.exe

Use the strings tool to list all human-readable strings of the AV-Check binary. What is the flag?

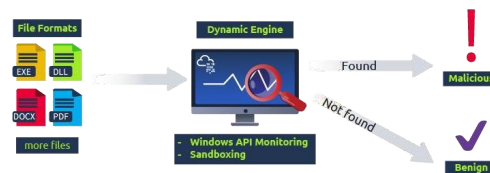
Answer: THM{Y0uC4nC-5tr16s}

Task 6: Other Detection Techniques

The concept of static detection is relatively simple. In this section, we will discuss the different types of detection techniques.

Dynamic Detection

The dynamic detection approach is advanced and more complicated than static detection. Dynamic detection is focused more on checking files at runtime using different methods. The following diagram shows the dynamic detection scanning flow:



The first method is by monitoring Windows APIs. The detection engine inspects Windows application calls and monitors Windows calls using Windows [Hooks \(opens in new tab\)](#).

Another method for dynamic detection is Sandboxing. A is a virtualized environment used to run malicious files separated from the host computer. This is usually done in an isolated environment, and the primary goal is to analyze how the malicious software acts in the system. Once the malicious software is confirmed, a unique signature and rule will be created based on the characteristic of the binary. Finally, a new update will be pushed into the cloud database for future use.

This type of detection also has drawbacks because it requires executing and running the malicious software for a limited time in the virtual environment to protect the system resources. As with other detection techniques, dynamic detection can be bypassed. Malware developers implement their software to not work within the virtual or simulated environment to avoid . For example, they check if the system spawns a real process of executing the software before running malicious activities or let the software wait sometime before execution.

For more information about evasion, we suggest checking the room: [Evasion!](#)

Heuristic and Behavioral Detection

Heuristic and behavioral detection have become essential in today's modern products. Modern software relies on this type of detection to detect malicious software. The heuristic analysis uses various techniques, including static and dynamic heuristic methods:

1. Static Heuristic Analysis is a process of decompiling (if possible) and extracting the source code of the malicious software. Then, the extracted source code is compared to other well-known virus source codes. These source codes are previously known and predefined in a heuristic database. If a match meets or exceeds a threshold percentage, the code is flagged as malicious.
2. Dynamic Heuristic Analysis is based on predefined behavioral rules. Security researchers analyzed suspicious software in isolated and secured environments. Based on their findings, they flagged the software as malicious. Then, behavioral rules are created to match the software's malicious activities within a target machine.

The following are examples of behavioral rules:

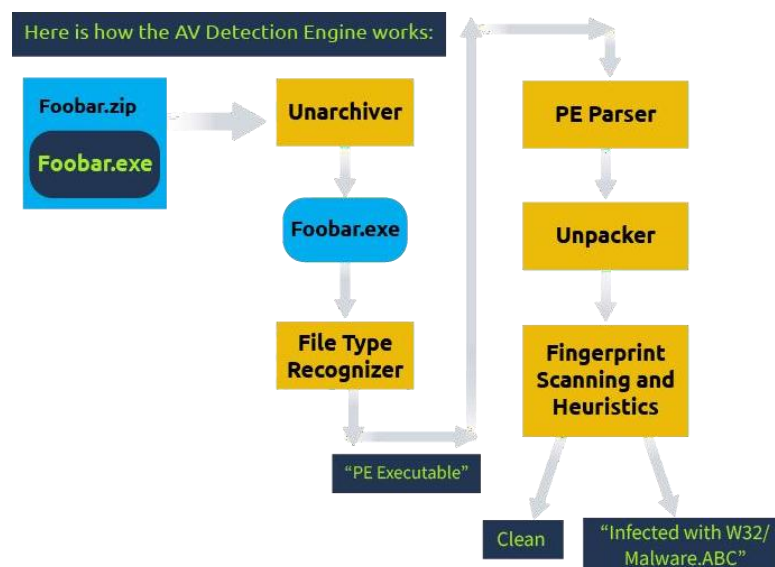
- If a process tries to interact with the LSASS.exe process that contains users' hashes, tickets, and more
- If a process opens a listening port and waits to receive commands from a Command and Control () server

The following diagram shows the Heuristic and behavioral detection scanning flow:



Summing up detection methods

Let's summarize how modern software works as one unit, including all components, and combines various features and detection techniques to implement its engine. The following is an example of the components of an antivirus engine:



In the diagram, you can see a suspicious **Fooobar.zip** file is passed to software to scan. software recognizes that it is a compressed file (.zip). Since the software supports .zip files, it applies an un-archiver feature to extract the files (**Fooobar.exe**). Next, it identifies the file type to know which module to work with and then performs a parsing operation to pull the binary's information and other characteristic features. Next, it checks whether the file is packed; if it is, it unpacks the code. Finally, it passes the collected information and the binary to the engine, where it tries to detect if it is malicious and gives us the result.

Answer the questions below

Which detection method is used to analyze malicious software inside virtual environments?

Answer: Dynamic Detection

Task 7 AV Testing and Fingerprinting

Vendors

Many vendors in the market mainly focus on implementing a security product for home or enterprise users. Modern software has improved and now combines antivirus capabilities with other security features such as , Encryption, Anti-spam, , vulnerability scanning, , etc.

It is important to note that it is hard to recommend which software is the best. It all comes down to user preferences and experience. Nowadays, vendors focus on business security in addition to end-user security. We suggest checking the [comparatives website \(opens in new tab\)](#) for more details on enterprise vendors.

Testing Environment

testing environments are a great place to check suspicious or malicious files. You can upload files to get them scanned against various software vendors. Moreover, platforms such as VirusTotal use various techniques and provide results within seconds. As a red teamer or a pentester, we must test a payload against the most well-known applications to check the effectiveness of the bypass technique.

VirusTotal



VirusTotal is a well-known web-based scanning platform for checking suspicious files. It allows users to upload files to be scanned with over 70 antivirus detection engines. VirusTotal passes the uploaded files to the Antivirus engines to be checked, returns the result, and reports whether it is malicious or not. Many checkpoints are applied, including checking for blacklisted URLs or services, signatures, binary analysis, behavioral analysis, as well as checking for calls. In addition, the binary will be run and checked in a simulated and isolated environment for better results. For more information and to check other features, you may visit the [VirusTotal \(opens in new tab\)](#) website.

AV VirusTotal alternatives

Important Note: VirusTotal is a handy scanning platform with great features, but it has a sharing policy. All scanned results will be passed and shared with antivirus vendors to improve their products and update their databases for known malware. As a red teamer, this

will burn a dropper or a payload you use in engagements. Thus, alternative solutions are available for testing against various security product vendors, and the most important advantage is that they do not have a sharing policy. However, there are other limitations. You will have a limited number of files to scan per day; otherwise, a subscription is needed for unlimited testing. For those reasons, we recommend you only test your malware on sites that do not share information, such as:

- [AntiscanMe \(opens in new tab\)](#) (6 free scans a day)
- [Virus Scan Jotti's malware scan \(opens in new tab\)](#)

Fingerprinting software

As a red teamer, we do not know what software is in place once we gain initial access to a target machine. Therefore, it is important to find and identify what host-based security products are installed, including software. fingerprinting is an essential process to determine which vendor is present. Knowing which software is installed is also quite helpful in creating the same environment to test bypass techniques.

This section introduces different ways to look at and identify antivirus software based on static artifacts, including service names, process names, domain names, registry keys, and filesystems.

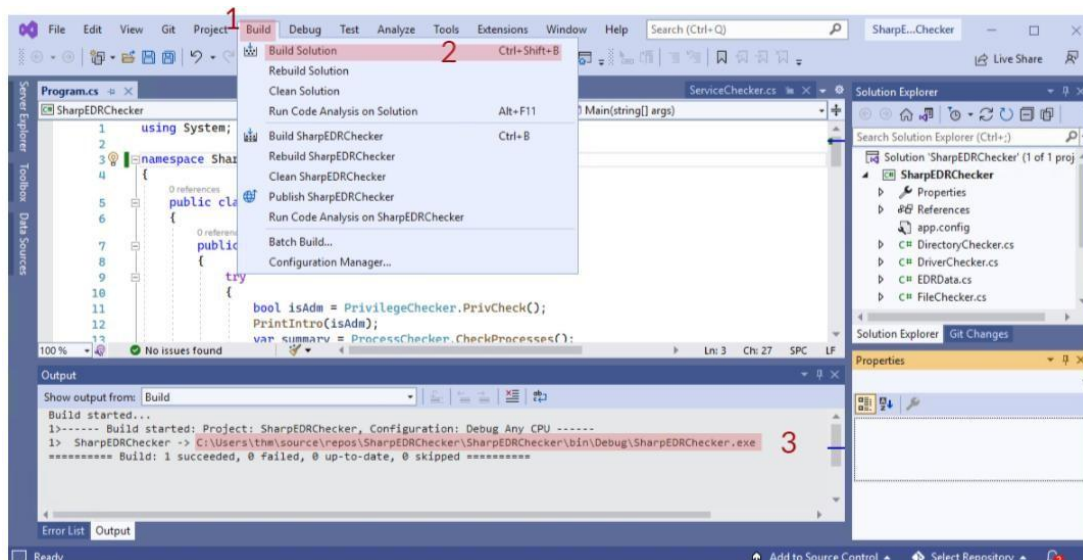
The following table contains well-known and commonly used software.

Antivirus Name	Service Name	Process Name
Microsoft Defender	WinDefend	MSMpEng.exe
Trend Micro	TMBMSRV	TMBMSRV.exe
Avira	AntivirService, Avira.ServiceHost	avguard.exe, Avira.ServiceHost.exe
Bitdefender	VSSERV	bdagent.exe, vsserv.exe
Kaspersky	AVP<Version #>	avp.exe, ksde.exe
AVG	AVG Antivirus	AVGSvc.exe
Norton	Norton Security	NortonSecurity.exe
McAfee	McAPExe, Mfemms	McAPExe.exe, mfemms.exe
Panda	PavPrSvr	PavPrSvr.exe
Avast	Avast Antivirus	afwServ.exe, AvastSvc.exe

SharpEDRChecker

One way to fingerprint is by using public tools such as [SharpEDRChecker \(opens in new tab\)](#). It is written in C# and performs various checks on a target machine, including checks for software, like running processes, files' metadata, loaded files, Registry keys, services, directories, and files.

We have pre-downloaded the SharpEDRChecker from the [GitHub repo \(opens in new tab\)](#) so that we can use it in the attached . Now we need to compile the project, and we have already created a shortcut to the project on the desktop (SharpEDRChecker). To do so, double-click on it to open it in Microsoft Visual Studio 2022. Now that we have our project ready, we need to compile it, as shown in the following screenshot:



Once it is compiled, we can find the path of the compiled version in the output section, as highlighted in step 3. We also added a copy of the compiled version in the **C:\Users\Desktop\Files** directory. Now let's try to run it and see the result as follows:

Command Prompt!	SharpEDRChecker's Summary
C:\>	[!] Directory Summary:
SharpEDRChecker.exe	[-] C:\Program Files\Windows Defender : defender
	[-] C:\Program Files\Windows Defender Advanced Threat Protection : defender, threat
	[-] C:\Program Files (x86)\Windows Defender : defender
	→ [!] Service Summary:
	[-] PsShutdownSvc : sysinternal
	[-] Sense : defender, threat
	[-] WdNisSvc : defender, nissrv
	[-] WinDefend : antimalware, defender, malware, msmppeng
	[-] wscsvc : antivirus

As a result, the Windows Defender is found based on folders and services. Note that this program may be flagged by AV software as malicious since it does various checks and APIs calls.

C# Fingerprint checks

Another way to enumerate AV software is by coding our own program. We have prepared a C# program in the provided Windows 10 Pro VM, so we can do some hands-on experiments! You can find the project's icon on the desktop (AV-Check) and double-click it to open it using Microsoft Visual Studio 2022.

The following C# code is straightforward, and its primary goal is to determine whether AV software is installed based on a predefined list of well-known AV applications.

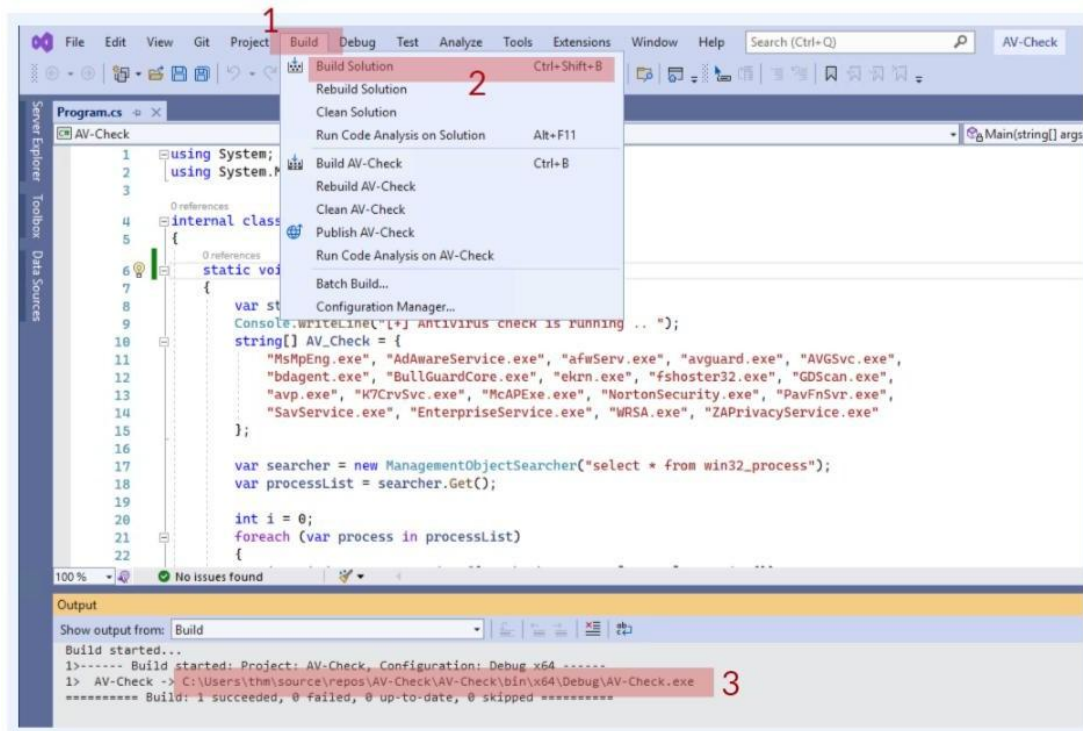
```
using System;using System.Management;
internal class Program{
    static void Main(string[] args)
    {
```

```
var status = false;
Console.WriteLine("[+] Antivirus check is running .. ");
string[] AV_Check = {
    "MsMpEng.exe", "AdAwareService.exe", "afwServ.exe", "avguard.exe",
"AVGSvc.exe",
    "bdagent.exe", "BullGuardCore.exe", "ekrn.exe", "fshoster32.exe", "GDScan.exe",
    "avp.exe", "K7CrvSvc.exe", "McAPExe.exe", "NortonSecurity.exe", "PavFnSvr.exe",
    "SavService.exe", "EnterpriseService.exe", "WRSA.exe", "ZAPrivacyService.exe"
};
var searcher = new ManagementObjectSearcher("select * from win32_process");
var processList = searcher.Get();
int i = 0;
foreach (var process in processList)
{
    int _index = Array.IndexOf(AV_Check, process["Name"].ToString());
    if (_index > -1)
    {
        Console.WriteLine("--AV Found: {0}", process["Name"].ToString());
        status = true;
    }
    i++;
}
if (!status) { Console.WriteLine("--AV software is not found!"); }
}}
```

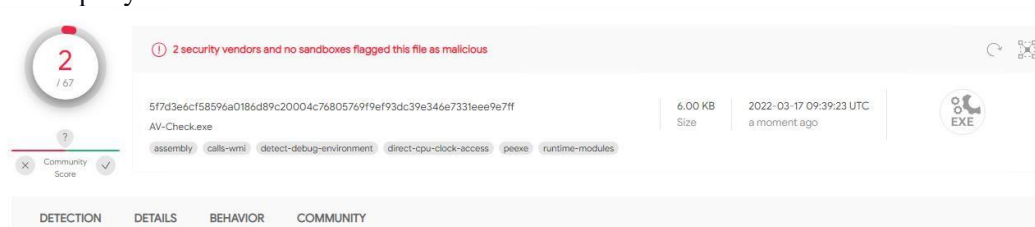
Let's explain the code a bit more. We have predefined a list of well-known applications in the **AV_Check** array within our code, which is taken from the previous section, where we discussed fingerprinting software (table above). Then, we use the Windows Management Instrumentation Command-Line (WMIC) query (**select * from win32_process**) to list all currently running processes in the target machine and store them in the **processList** variable. Next, we go through the currently running processes and compare if they exist in the predefined array. If a match is found, then we have software installed.

The C# program utilizes a WMIC object to list current running processes, which may be monitored by software. If software is poorly implemented to monitor the WMIC queries or Windows APIs, it may cause false-positive results in scanning our C# program.

Let's compile an x86 version of the C# program, upload it to the VirusTotal website, and check the results! To compile the C# program in the Microsoft Visual Studio 2022, select **Build** from the bar menu and choose the **Build Solution** option. Then, if it compiled correctly, you can find the path of the compiled version in the output section, as highlighted in step 3 in the screenshot below.



If we upload the -Check program to the [VirusTotal website \(opens in new tab\)](#) and check the result, surprisingly, VirusTotal showed that two vendors (MaxSecure and SecureAge APEX) flagged our program as malicious! Thus, this is a false-positive result where it incorrectly identifies a file as malicious where it is not. One of the possible reasons is that these vendors' software uses a machine-learning classifier or rule-based detection method that is poorly implemented. For more details about the actual submission report, see [here \(opens in new tab\)](#). There are four main sections: Detection, Details, Behavior, and Community. If we check the Behavior section, we can see all calls of Windows APIs, Registry keys, modules, and the WMIC query.



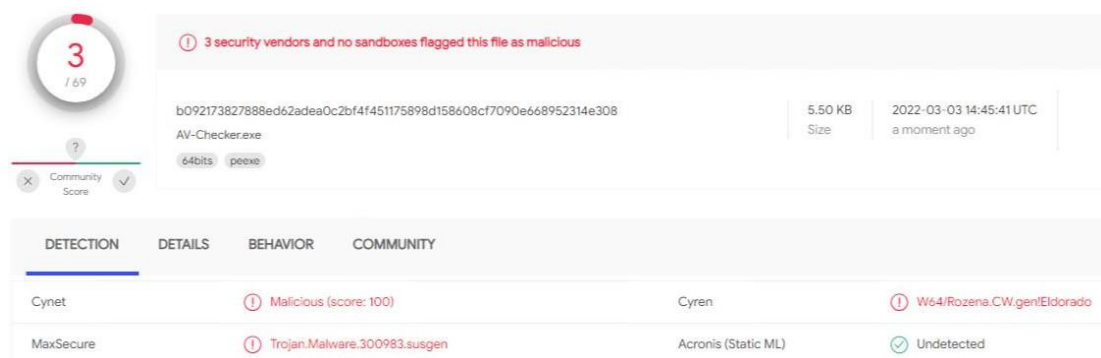
In the Detection section, there are Sigma rules that, if a system event during execution is matched (in the environment), consider the file malicious. This result is likely based on the rules; VirusTotal flagged our program because of the [Process Ghosting technique \(opens in new tab\)](#), as shown in the following screenshot.

Crowdsourced Sigma Rules ⓘ

CRITICAL 0 HIGH 2 MEDIUM 0 LOW 28

- ⚠️ 1 match for rule **Execution of Suspicious File Type Extension** by Max Altgelt from Sigma Integrated Rule Set (GitHub)
Checks whether the image specified in a process creation event doesn't refer to an .exe file (caused by process ghosting or other unorthodox methods to start a process)
- ⚠️ 1 match for rule **Execution Of Non-Existing File** by Max Altgelt from Sigma Integrated Rule Set (GitHub)
Checks whether the image specified in a process creation event is not a full, absolute path (caused by process ghosting or other unorthodox methods to start a process)
- 🔍 28 matches for rule **Stop Windows Service** by Jakob Weinzettl, oscd.community from Sigma Integrated Rule Set (GitHub)
↳ Detects a windows service to be stopped

Now let's re-compile the C# program using an x64 and check if the check engines act differently. In our submission attempt this time, three vendors' software (Cyren is added to the list) flagged the file as malicious. For more details about the actual submission report, look [here \(opens in new tab\)](#).



The image shows a VirusTotal submission report for a file named 'AV-Checker.exe'. The file has a SHA-256 hash of b092173827888ed62adea0c2bf4f451175898d158608cf7090e668952314e308, is 5.50 KB in size, and was uploaded on 2022-03-03 14:45:41 UTC. The report shows a community score of 3/69. The detection results are as follows:

DETECTION	DETAILS	BEHAVIOR	COMMUNITY	
Cynet	⚠️ Malicious (score: 100)		Cyren	⚠️ W54/Rozena.CW/gen/Eldorado
MaxSecure	⚠️ Trojan.Malware.300983.susgen		Acronis (Static ML)	✅ Undetected

Note: if you try to submit a file to the VirusTotal website, it may give you a different result. Keep in mind that VirusTotal shares submission reports with the Antivirus vendors to improve their detection engines, including false-positive results.

Answer the questions below

For the C# AV fingerprint, try to rewrite the code in a different language, such as Python, and check whether VirusTotal flag it as malicious.

No Answer Needed

Read the Above!

No Answer Needed

Task 8: Conclusion

Recap

In this room, we covered Antivirus software and its detection approaches. As a red teamer, it is important to know how software works and detects malicious applications to be able to implement bypass techniques.

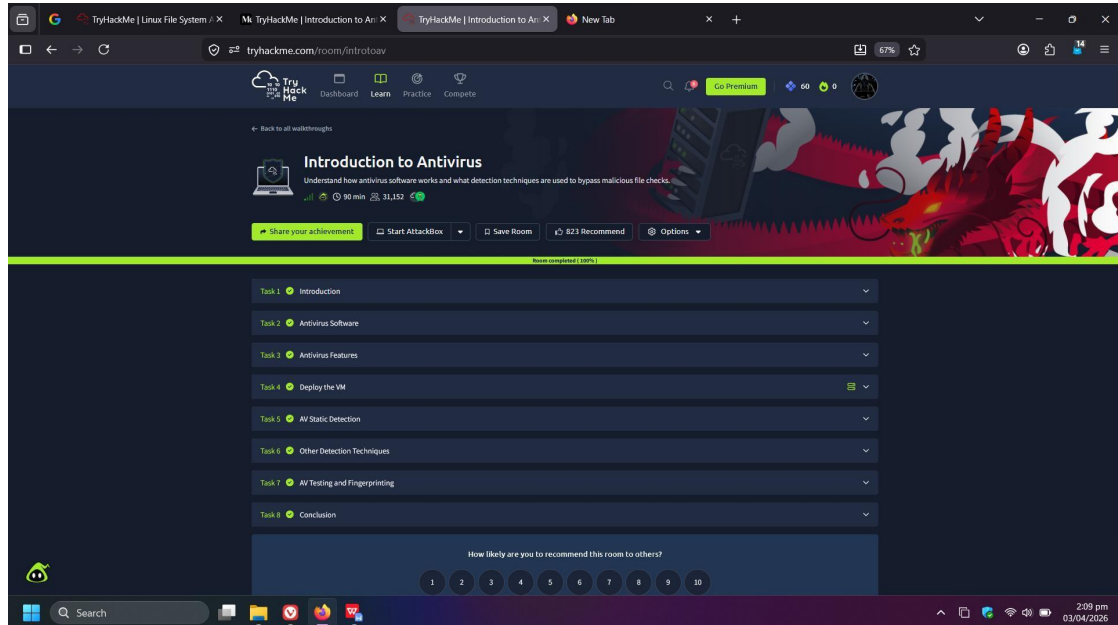
We also discussed the static detection technique in detail and showcased how ClamAV, an open-source Antivirus, detects malicious files using . Additionally, we showed how to create your own database and use Yara rules to detect malicious files that are not detected by the official database.

Once we obtain access to a target, it is essential to enumerate the target machine before performing further actions, such as privilege escalation or lateral movement. The reason is to not trigger alerts for suspicious activities, which may cause losing access to the target machine. Thus, we introduced two methods to practice fingerprinting using public and private tools.

Answer the questions below

Congrats on completing the room, and keep learning!

No Answer Needed



Result:

Successfully learned how antivirus detects malicious files using signatures, behavior, and sandboxing, and practiced identifying and testing malware using tools like ClamAV and Yara rules.